

CMSC 313 Project 1

Name: _____

Assigned: 9/16/2026

Due: 9/28/2026 at 11:59pm

Project Description

In this project, you will design a 4-bit Arithmetic Logic Unit (ALU) using Logisim Evolution. A skeleton file named Project1_Skeleton.circ will help you get started – make sure to download it from Blackboard. Please **rename the skeleton file** to Project1_YOURLASTNAME.circ before submitting the project to Blackboard.

Do **NOT** rename any of the input/output pins. Do **NOT** change the vertical order of any input pins. Do **NOT** change the vertical order of any output pins. You are allowed to add labels to the logic gates in your circuit, but you are not required to. **Failing to follow these instructions will be a 50 point deduction from this assignment's grade.**

Arithmetic Logic Unit

As discussed in class, an ALU is a digital circuit that can carry out several arithmetic and bitwise operations. The ALU you will design operates on two 4-bit integer inputs (A and B). It returns a 4-bit integer output (Q). It will perform eight different operations, and the ALU will select which operation to perform according to the 3-bit opcode given to it.

Opcode	Operation
000	$Q = B$ (No_Operation)
001	$Q = A = B$ (Equality_Comparison)
010	$Q = A \text{ XOR } B$ (Bitwise_XOR)
011	$Q = \overline{B}$ (Inverter)
100	$Q = B + 1$ (Increment)
101	$Q = A + B$ (Addition)
110	$Q = A - B$ (Subtraction)
111	$Q = A * B$ (Multiplication)

ALU Inputs (A, B, and Opcode)

A (3:0) – 4 bits of input to be operated on ($A_3 A_2 A_1 A_0$).

B (3:0) – 4 bits of input to be operated on ($B_3 B_2 B_1 B_0$).

Opcode (2:0) – 3 bits of input indicating the operation the ALU should perform ($O_2 O_1 O_0$)

ALU Output (Q)

Q (3:0) – 4 bits of output that are the result of the operation ($Q_3 Q_2 Q_1 Q_0$)

Subcircuits:

Each of the following should be implemented as its own subcircuit in Logisim Evolution. You may use any combination of the following components in Logisim Evolution to implement them: NOT, AND, OR, NAND, NOR, XOR, XNOR. Using any of the components in the “Arithmetic” folder in Logisim Evolution will result in a 0, since they trivialize the assignment.

No_Operation – This circuit has been completed for you. It sets Q equal to B. The input A is ignored.

Equality_Comparison – This circuit should compare A and B. If they are equal, Q_0 should be set to 1. If they are not equal, Q_0 should be set to 0. The skeleton file sets Q_1 , Q_2 , and Q_3 to 0 – do not alter this.

Bitwise_XOR – This should set Q equal to the result of A XOR B. That is, $Q_0 = A_0 \text{ XOR } B_0$, $Q_1 = A_1 \text{ XOR } B_1$, and so on.

Inverter – This should set Q equal to $\overline{B}$. That is, $Q_0 = \overline{B_0}$, $Q_1 = \overline{B_1}$, and so on.

Increment – This should set Q equal to B plus 1. The input A is ignored. Be aware that A and B may be positive or negative (with negative numbers being represented in two’s complement format). Use Half Adders (but no Full Adders) in your implementation of Increment.

Addition – This should set Q equal to the result of A plus B using a ripple-carry adder.

Subtraction – This should set Q equal to the result of A minus B. Your implementation should convert B to its two’s complement representation (invert the bits of B and add 1) and then add it to A. This is equivalent to $A + (-B)$.

Multiplication – This should set Q equal to the lowest 4 bits of A multiplied by B. You may assume that A and B are unsigned 4-bit integers. You do not need to implement multiplication for bits 5-8 of the product. The multiplier should be made only from AND gates, Half adders, and Full adders.

main - The Arithmetic Logic Unit (ALU) should be implemented inside of main. Add subcircuits for each of the 8 operations above and connect each of them to A and B. The ALU will evaluate each of the 8 operations. Depending on the Opcode input, one of these operations will be selected, and the output of

that operation will be forwarded to Q. Make sure that each opcode maps to the correct operation (see the table on page 1).

Other Notes and Tips

- You can use a “constant” inside of the Wiring folder in Logisim Evolution to set a value to always 1 or always 0.
- If it helps make your design simpler, you are free to include one subcircuit inside of another subcircuit.
- For simplicity, do not worry about detecting overflow in any of the operations. You may discard the carry outputs of any adders that would normally be used for overflow detection.

How to submit:

Test your circuit with various inputs to make sure that your ALU and all of your operations work properly. You are encouraged to test as you go. Please **rename the skeleton file** to Project1_YOURLASTNAME.circ before submitting the project to Blackboard. Then, submit your completed Project1_YOURLASTNAME.circ file on Blackboard.